



UTEC Posgrado

Feedback y Auto-corrección en Agentes LLM

Cómo los agentes aprenden de sus
propios errores

Desarrollo de Agentes IA

UTEC
Universidad
de Ingeniería
y Tecnología

Contenido

El problema

¿De dónde viene la señal?

Interno: el agente se juzga solo

Externo, multi-agente y humano

Los métodos, uno por uno

¿Funciona de verdad?

La letra pequeña

De corregir a mejorarse

Práctica

Cierre

Dónde estamos

El problema

¿De dónde viene la señal?

Los métodos, uno por uno

¿Funciona de verdad?

De corregir a mejorarse

Práctica

Cierre

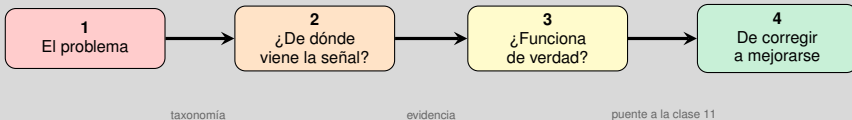
Objetivos

Al final de esta sesión podrás:

- **Clasificar** mecanismos de feedback en agentes LLM
- **Distinguir** cuándo la auto-corrección funciona y cuándo no
- **Evaluar** agentes con métricas de resultado y de proceso
- **Implementar** loops de auto-corrección con LangGraph

Todo parte de una observación incómoda: un LLM puede fallar mil veces igual.

El recorrido de hoy

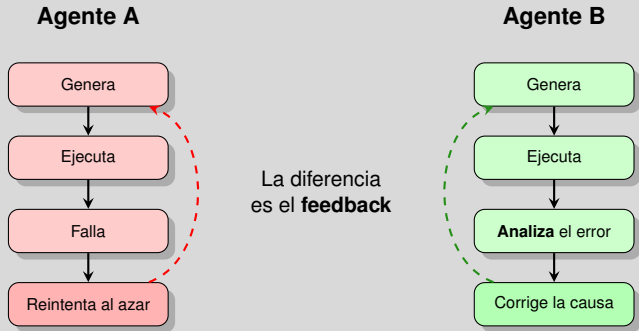


La pregunta que nos guía

Un agente acaba de fallar. **¿Qué tiene que pasar para que la próxima vez no falle igual?**

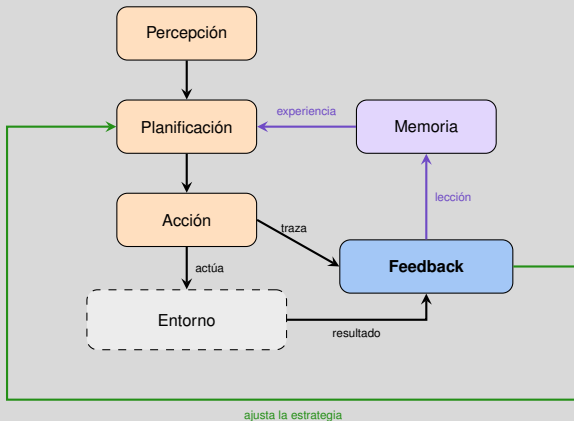
Empecemos viendo qué separa a un agente que aprende de uno que solo reintent.

Dos formas de fallar



Si el feedback es lo que los separa, ¿dónde vive dentro de la arquitectura del agente?

El feedback es un módulo, no un truco de prompt



Acoplado a memoria y planificación: por eso una corrección puede volverse aprendizaje.

Marco unificado de 5 módulos: Liu et al., IJCAI-25, §2 (Fig. 1).

Discusión en grupo - 3 min

Piensen en una tarea concreta que le darían a un agente

1. ¿Cómo se daría cuenta **el agente** de que se equivocó?
2. ¿Cómo se darían cuenta **ustedes**? ¿Tardarían minutos o semanas?
3. Si nadie puede detectar el error, ¿tiene sentido automatizar esa tarea?

Guarden su respuesta

Volveremos a esta tarea en cada discusión de la clase.

Dónde estamos

El problema

¿De dónde viene la señal?

Interno: el agente se juzga solo

Externo, multi-agente y humano

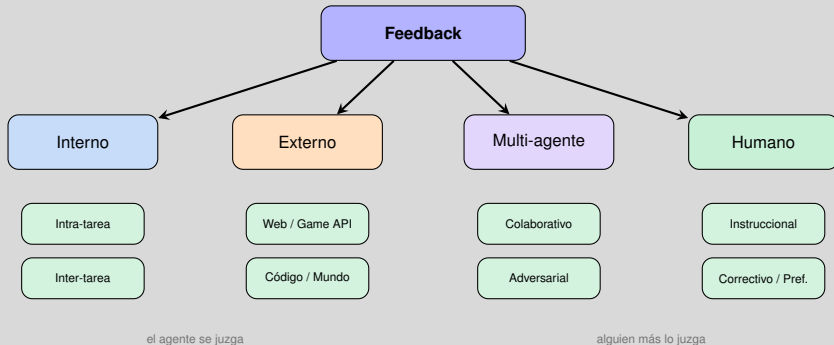
Los métodos, uno por uno

¿Funciona de verdad?

De corregir a mejorarse

Práctica

Cuatro respuestas posibles

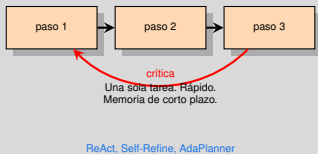


Recorreremos las cuatro en ese orden: de la más barata y menos confiable, a la más costosa y más confiable.

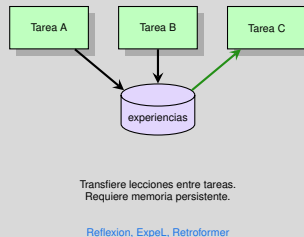
Interno: intra-tarea vs inter-tarea

Fuente 1 de 4. Aquí no entra nadie más: la señal la produce el propio modelo.

Intra-tarea



Inter-tarea



Liu et al., IJCAI-25, §3.1 y Tabla 1.

Esa memoria persistente de la derecha es exactamente el tema de la clase 11. Guárdalo.

Interno: el trade-off

Aspecto	Intra-tarea	Inter-tarea
Alcance	Tarea actual	Múltiples tareas
Velocidad	Rápida	Más lenta
Memoria	Corto plazo	Largo plazo
Generalización	Limitada	Alta
Ejemplos	ReAct, Self-Refine	Reflexion, ExpeL

Pregunta abierta

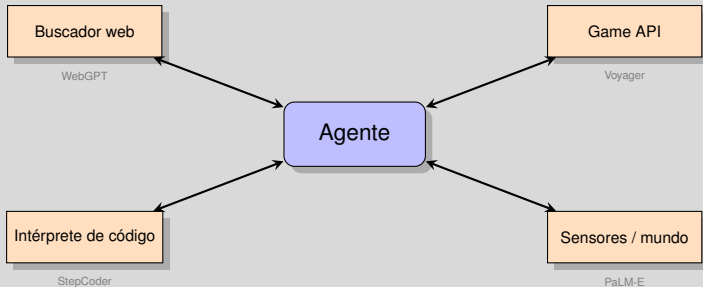
¿Cómo combinar ambos sin que la memoria crezca sin control?

Síntesis de la discusión de Liu et al., IJCAI-25, §3.1.

*Nota el punto débil común: en los dos casos **el juez es el propio modelo**. Cambiemos eso.*

Externo: que responda el mundo

Fuente 2 de 4. La señal ya no la inventa el modelo.



La señal es **verificable**: el compilador no alucina.

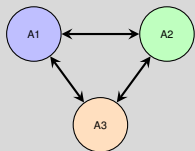
Liu et al., IJCAI-25, §3.2.

Recuerda esta frase: al final de la clase será la que decida si tu loop sirve o no.

Multi-agente: que respondan otros agentes

Fuente 3 de 4. Si nadie tiene la verdad, al menos que haya varias perspectivas.

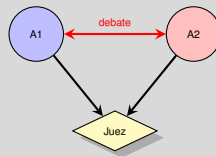
Colaborativo



MetaGPT, InteRecAgent

Roles complementarios

Adversarial



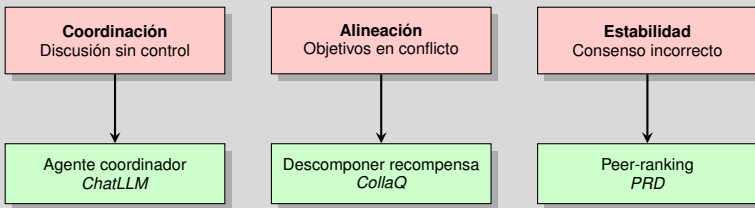
Multiagent Debate, ChatEval

Consenso por argumentación

Liu et al., IJCAI-25, §3.3.

Poner más agentes suena bien... hasta que empiezan a estorbarse. Tres problemas conocidos:

Multi-agente: tres problemas, tres soluciones

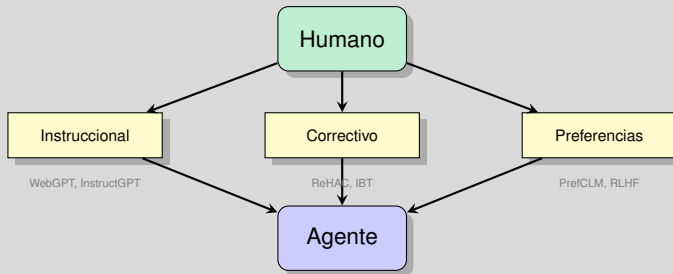


Problemas y soluciones citados en Liu et al., IJCAI-25, §3.3 (discusión).

Estos tres problemas son el esqueleto de la clase 12. Hoy solo los nombramos.

Humano: la señal más confiable y la más cara

Fuente 4 de 4.



No escala

Requiere monitoreo continuo. La salida habitual: **LLM-as-a-judge** → pero eso nos devuelve al problema del juez poco confiable.

Liu et al., IJCAI-25, §3.4. LLM-as-a-judge: Zheng et al., 2023. Agent-as-a-judge: Zhuge et al., 2024.

Y además arrastra los sesgos de quien anota

El costo oculto del feedback humano

Sesgos

- Género, raza, cultura en anotadores
- Se **amplifican** en el modelo
- Poca diversidad en los datasets

Mitigación

- Datasets culturalmente representativos
- Anotadores de orígenes diversos
- Auditorías de equidad

Riesgos y mitigaciones señalados en Liu et al., IJCAI-25, §3.4 (discusión ética).

Con esto cerramos las cuatro fuentes. Antes de seguir, ordenémoslas por lo que de verdad importa.

Recap: las cuatro fuentes ordenadas

Fuente	¿Quién juzga?	Confiabilidad	Costo
Interno	el propio modelo	Baja	Bajo
Externo	el entorno	Alta	Medio
Multi-agente	otros modelos	Media	Alto
Humano	una persona	Alta	Muy alto

La tensión de todo el diseño

Lo barato no es confiable; lo confiable no es barato.

Tabla propia, sintetizada de las discusiones de Liu et al., IJCAI-25, §3.1–3.4.

Antes de comparar resultados, veamos qué hace realmente cada método.

Discusión en grupo · 3 min

Vuelvan a su tarea: ¿de dónde sacarían la señal?

1. ¿Cuál de las cuatro fuentes usarían? ¿Cuál es **imposible** en su caso y por qué?
2. ¿Cuánto costaría la más confiable que sí pueden conseguir?
3. ¿Podrían combinar dos fuentes: una barata para filtrar y una cara para confirmar?

Pista

Casi siempre hay una señal externa escondida: un log, un test, una base de datos, un cálculo que ya existe.

Dónde estamos

El problema

¿De dónde viene la señal?

Los métodos, uno por uno

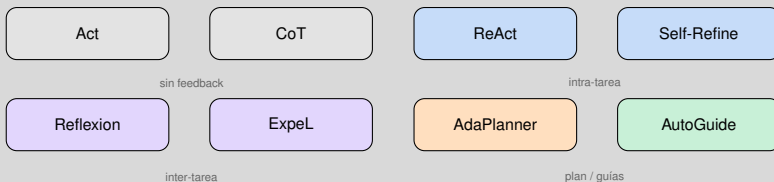
¿Funciona de verdad?

De corregir a mejorarse

Práctica

Cierre

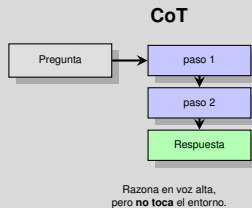
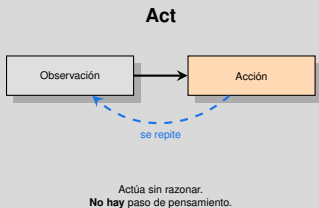
Ocho métodos que veremos comparados



Las tablas de la próxima sección los comparan. Para que los números signifiquen algo, primero hay que saber **qué hace cada uno**.

*Empecemos por los dos que **no** usan feedback: son la línea base.*

Act y CoT: las dos líneas base

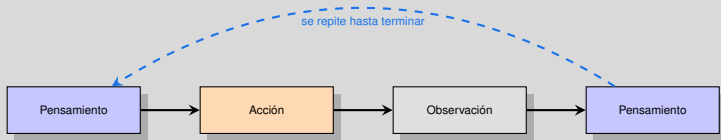


Act vs ReAct: la única diferencia es el paso de *Pensamiento*

Act es literalmente ReAct **sin** razonar. Comparando sus tasas de éxito: HotpotQA 29 % → 28 % (no ayuda) | WebShop 34 % → 35 % (apenas) | ALFWorld 28 % → **40 % (+12 puntos)**. Razonar paga en tareas **secuenciales largas**, no en preguntas de un solo salto.

Act y CoT como baselines: Yao et al., ICLR 2023 (ReAct); Wei et al., NeurIPS 2022 (CoT). Cifras: Liu et al., IJCAI-25, Tabla 3.

ReAct: razonar y actuar alternando



Qué aporta

El pensamiento decide la siguiente acción; la observación corrige el pensamiento.

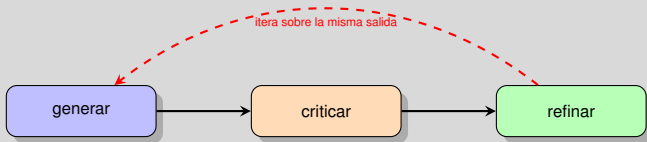
Qué le falta

Todo muere al acabar la tarea. **No guarda nada.**

Yao et al., “ReAct”, ICLR 2023. Clasificado como interno intra-tarea en Liu et al., IJCAI-25.

ReAct reacciona al entorno. ¿Y si el agente se critica a sí mismo?

Self-Refine: criticarse dentro de la misma tarea



el mismo LLM hace los tres papeles

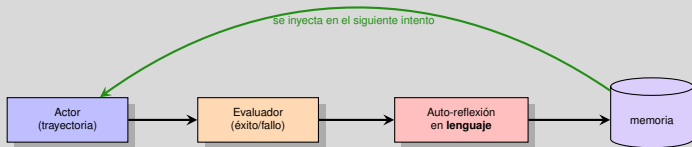
El punto débil

No hay verificador externo: si la crítica se equivoca, el refinamiento empeora la respuesta.

Madaan et al., "Self-Refine", NeurIPS 2023. Interno intra-tarea en Liu et al., IJCAI-25.

*Hasta aquí, todo se olvida al terminar. Los tres siguientes **recuerdan**.*

Reflexion: Guardar el fracaso



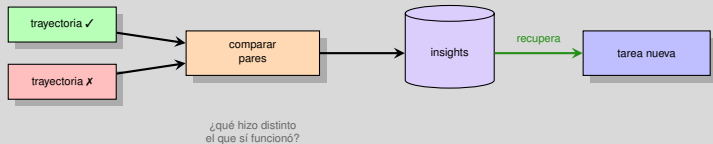
La idea clave

En vez de una recompensa numérica, el agente escribe **por qué** falló. Ese texto es más informativo que un escalar.

Shinn et al., "Reflexion", NeurIPS 2023. Interno inter-tarea en Liu et al., IJCAI-25.

*Reflexion aprende de **sus** fallos. ExpeL aprende de una colección entera.*

ExpeL: destilar reglas comparando éxitos/fracasos



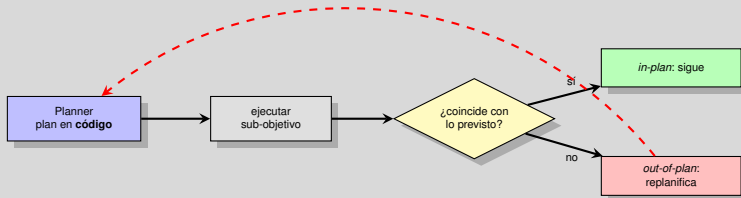
Diferencia con Reflexion

Reflexion reflexiona sobre *un* fallo. ExpeL extrae **patrones** de muchas trayectorias previas y los reutiliza.

Zhao et al., "ExpeL: LLM Agents are Experiential Learners", AAI 2024.

*Los dos últimos atacan el problema desde otro ángulo: el **plan** y las **guías**.*

AdaPlanner: el plan es código y se corrige solo



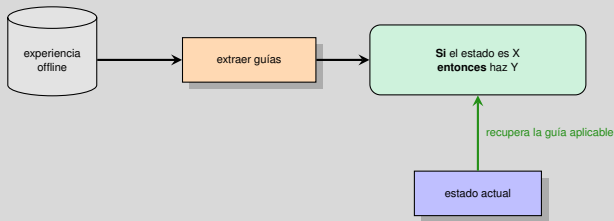
Por qué el código importa

El prompt en estilo código fuerza a descomponer en sub-objetivos y **reduce las alucinaciones** del planificador.

Sun et al., "AdaPlanner", NeurIPS 2023.

Y el último: en vez de replanificar, consultar una regla escrita de antemano.

AutoGuide: guías condicionadas al estado



La jugada

Comprime la experiencia en reglas breves en lenguaje natural, y solo inyecta en el prompt **la que aplica ahora.**

Fu et al., "AutoGuide", NeurIPS 2024.

Todo esto es hasta 2024. ¿Se detuvo ahí la investigación?

Después de 2024 el campo se parte en dos



Andamiaje (Σ) — son agentes

ERL (2026): reflexiona, guarda heurísticas con condición de disparo y las puntúa por relevancia antes de inyectarlas. El modelo no se toca.

Gaia2: **56.1 %** vs ReAct 48.3 %, ExpeL 50.9 %, AutoGuide 50.8 %

Modelo (θ) — no son agentes

SCoRe (2025) y AutoRefine (2025) son métodos de **post-entrenamiento de LLMs** con RL. No añaden andamiaje: enseñan al modelo a corregirse o a refinar lo que recupera.

ERL: Allard et al., arXiv:2603.24639. SCoRe: Kumar et al., ICLR 2025, Google DeepMind. AutoRefine: Shi et al., NeurIPS 2025, arXiv:2505.11277.

Vale la pena mirar los dos de la derecha: sus resultados dicen algo incómodo sobre todo lo anterior.

Los dos métodos que entrenan el modelo

SCoRe — corregirse

RL **multi-turno online** con datos enteramente auto-generados, sin oráculo ni modelo maestro.

Hallazgo previo: el SFT sobre trazas de corrección **no basta** (desajuste de distribución o colapso de conducta).

Dos fases: inicialización que no colapsa + bonus de recompensa que amplifica la corrección.

AutoRefine — refinar lo recuperado

Paradigma *search-and-refine-during-think*:

`<think> → <search> → <refine> → <answer>`

GRPO con **dos** recompensas: acierto de la respuesta + una recompensa específica sobre el bloque `<refine>`.

Qwen2.5-3B. Avg QA: **0.405** vs Search-R1 0.336 (+6,9 pts), sobre todo en multi-hop.

Los dos operan en la caja **azul** (θ). Si no puedes reentrenar el modelo, no son opciones para ti.

Y ahora el dato de SCoRe que obliga a releer toda la sección anterior.

Recap: los ocho en una línea

Método	Qué hace	Tipo de feedback
Act	Actúa sin razonar	Ninguno
CoT	Razona sin actuar	Ninguno
ReAct	Alterna pensamiento y acción	Interno intra-tarea
Self-Refine	Se critica y reescribe su salida	Interno intra-tarea
Reflexion	Escribe por qué falló y lo guarda	Interno inter-tarea
ExpeL	Destila reglas de éxitos vs fracasos	Interno inter-tarea
AdaPlanner	Replanifica cuando la realidad no cuadra	Externo (entorno)
AutoGuide	Recupera la guía que aplica al estado	Externo (offline)

Clasificación según Liu et al., IJCAI-25, Tabla 1.

Con esto en la cabeza, los porcentajes de la próxima sección se vuelven interpretables.

Discusión en grupo · 3 min

Elijan una tarea que un agente haría en su trabajo o tesis

1. ¿Cuál de los ocho métodos encaja mejor con esa tarea?
2. ¿Su entorno permite usar Reflexion? (¿pueden **guardar** experiencia entre ejecuciones?)
3. Si tuvieran que elegir entre AdaPlanner y AutoGuide, ¿qué los inclinaría?

Para compartir

Una frase: “Usaríamos _____ porque nuestra señal viene de _____.”

Dónde estamos

El problema

¿De dónde viene la señal?

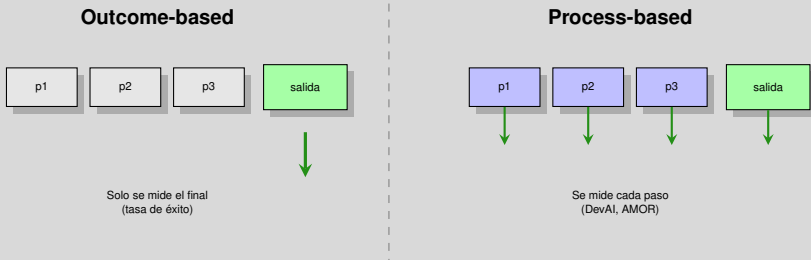
Los métodos, uno por uno

¿Funciona de verdad?
La letra pequeña

De corregir a mejorarse

Práctica

Primero: qué significa “medir” aquí



Hoy casi todo es outcome-based, y eso esconde **dónde** falló el agente.

Liu et al., IJCAI-25, §4.1. DevAI: Zhuge et al., 2024. AMOR: Guan et al., 2024.

Con esa advertencia en mente, veamos sobre qué se mide.

Sobre qué se mide

Dominio	Benchmark	Tamaño	Evaluación
Razonamiento	HotPotQA	113k pares Q&A	Outcome
	FEVER	185k claims	Outcome
Mundo virtual	ALFWorld	3.1k ejemplos	Outcome
	Minecraft	58 items	Outcome
Navegación web	WebShop	1.8k ej., 12k instrucciones	Outcome
	WebArena	812 tareas	Outcome
Código	HumanEval	164 problemas	Outcome
	SWE-Bench	2k problemas	Outcome
	DevAI	55 tareas	Process
Multi-agente	PARTNR	100k tareas, 5k objetos	Outcome

Tamaño: escala del conjunto de prueba (preguntas, tareas o problemas disponibles). **Evaluación:** *Outcome* = solo se puntúa la respuesta final, acertar o no; *Process* = se puntúa además cada paso intermedio del agente.

Liu et al., IJCAI-25, Tabla 2 — nota que **solo uno** evalúa el proceso.

Tomemos tres de estos benchmarks y comparemos los métodos que vimos.

Los números

Método	HotpotQA	ALFWorld	WebShop	Rondas
Act	29 %	28 %	34 %	1
CoT	29 %	—	—	Multi
ReAct	28 %	40 %	35 %	Multi
Reflexion-R1	33 %	48 %	43 %	1 + replay
Reflexion-R2	40 %	52 %	46 %	2 + replay
Reflexion-R3	40 %	54 %	48 %	3 + replay
ExpeL	39 %	59 %	41 %	Multi + replay
AdaPlanner	—	63 %	—	Multi
AutoGuide	—	79 %	46 %	Multi + replay

Las cifras son **tasa de éxito**: tareas resueltas sobre el total (más alto es mejor). **Rondas**: cuántas veces el agente vuelve a intentar aprovechando el feedback. *1* = un único intento, sin corregirse; *Multi* = varias iteraciones dentro de la misma tarea; *+ replay* = además reutiliza experiencia guardada de tareas anteriores. *R1/R2/R3* = 1, 2 o 3 reintentos de Reflexion. — = el método no se evaluó en ese benchmark.

Mira las filas R1 → R2 → R3: casi toda la ganancia está en la **primera** reflexión.

Liu et al., IJCAI-25, Tabla 3; algunos resultados tomados de ExpeL (Zhao et al., 2024) y AutoGuide (Fu et al., 2024).

El precio de reflexionar

Método	Costo comp.	Transferible	Tiempo real
Act / CoT	Bajo	Sí	✓
ReAct	Medio	Sí	✓
Reflexion	Alto	Requiere pool	✗
ExpeL	Alto	Requiere pool	✗
AdaPlanner	Medio	Sí	✓
AutoGuide	Alto	Requiere pool	✗

Costo comp.: llamadas al LLM y tokens que consume resolver una tarea. **Transferible:** si lo aprendido sirve en otra tarea o en otro agente; *requiere pool* = solo funciona si comparten el mismo almacén de experiencias. **Tiempo real:** ✓ responde lo bastante rápido para interactuar con un usuario; ✗ no.

El patrón

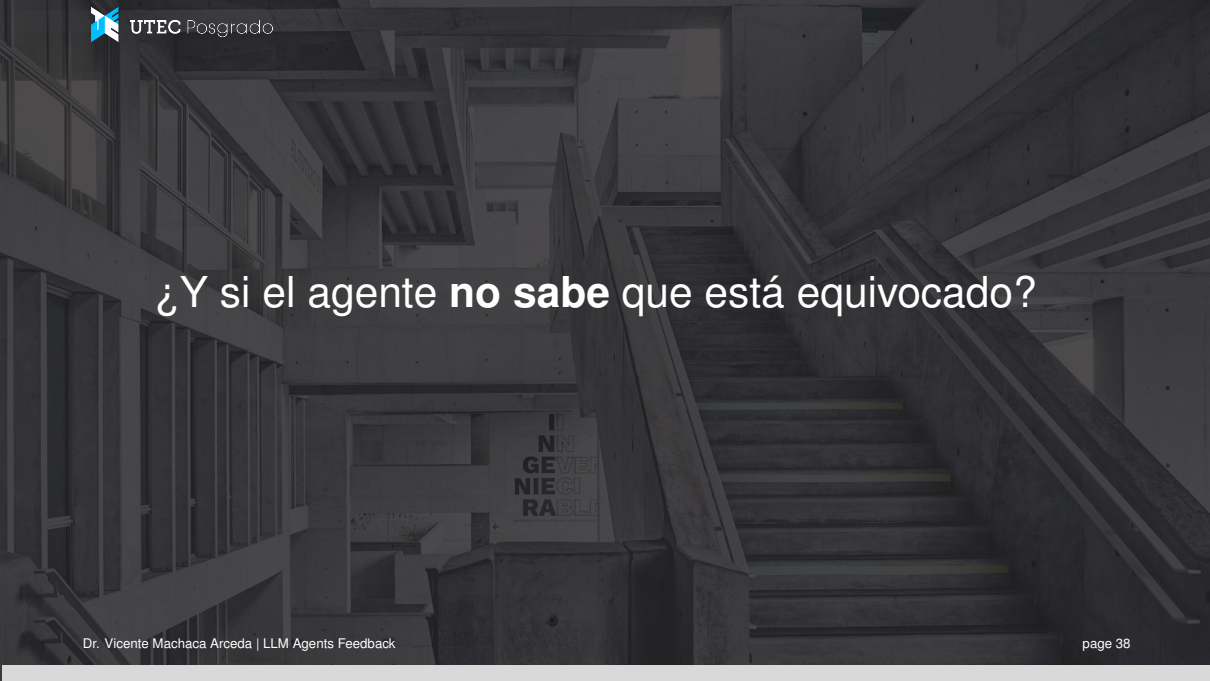
Todo lo que aprende de verdad sale del tiempo real y necesita un *pool* de experiencia.

La excepción

AdaPlanner: prompts en estilo código, sin sacrificar latencia.

Liu et al., IJCAI-25, Tabla 3 (columnas de costo, estabilidad y transferibilidad).

Hasta aquí, la historia optimista. Ahora la letra pequeña.



¿Y si el agente **no sabe** que está equivocado?

El hallazgo incómodo

Feedback de un LLM promptado
(auto-evaluación sin verificador)



Ningún trabajo previo lo
demuestra de forma limpia

Feedback externo confiable
(tests, compilador, DB)



Funciona de forma
consistente

Muchos estudios **sobreestiman** la auto-corrección: usan oráculos para decidir cuándo parar.

Nota que la columna verde es la “fuente externa” que vimos antes. Todo encaja.

Kamoi et al., TACL 2024

Auto-corregirse puede *empeorar* el resultado

Método (MATH)	Acc.@t1	Acc.@t2	$\Delta(t1,t2)$
Gemini 1.5 Flash (base)	52.6 %	41.4 %	-11,2 %
Self-Refine	52.8 %	51.8 %	-1,0 %
SCoRe (entrenado)	60.0 %	64.4 %	+4,4 %

Acc.@t1: acierto en el primer intento. **Acc.@t2**: acierto tras revisarse. $\Delta(t1,t2)$: cuánto gana el modelo por corregirse. Si es negativo, revisarse lo daña.

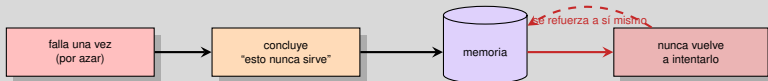
Lo que dice esta tabla

Pedirle a un modelo que se revise, sin más, le hace **perder 11 puntos**: cambia respuestas correctas por incorrectas. Self-Refine apenas frena el daño. Solo el modelo **entrenado para ello** obtiene una ganancia positiva, y es de +4,4.

Kumar et al., SCoRe, ICLR 2025, Tabla 2. Confirma empíricamente la tesis de Kamoi et al.

*Hay un segundo riesgo, más silencioso, cuando la reflexión además se **guarda**.*

El riesgo de recordar una conclusión falsa



Error auto-reforzado

Si la reflexión diagnóstica mal **y** esa conclusión se guarda, el agente deja de explorar una vía que sí funcionaba. La memoria convierte un error puntual en uno permanente.

“Honest Lying: Understanding Memory Confabulation in Reflexive Agents”, arXiv:2605.29463 (2026).

Dos advertencias, entonces. ¿Cómo se traducen en una regla de trabajo?

Checklist antes de confiar en tu loop

Pregunta siempre:

1. ¿De dónde viene la señal? ¿Es **verificable**?
2. ¿Quién decide **cuándo parar**? ¿Se usa la respuesta correcta para decidirlo?
3. ¿El baseline recibió el **mismo presupuesto** de cómputo?
4. ¿Mediste el Δ **con signo**? Una segunda pasada puede restar.
5. Si guardas la lección: ¿puedes **revocarla** cuando resulte falsa?

Regla práctica

Si no tienes verificador externo, invierte en **mejores prompts** antes que en más iteraciones.

Puntos 1–3 adaptados de Kamoi et al., TACL 2024; punto 4 de Kumar et al., ICLR 2025; punto 5 de arXiv:2605.29463.

Esa frase — “mejores prompts” — abre la puerta a la última parte de la clase.

Discusión en grupo - 3 min

Auditen su propio loop

1. Para la tarea que eligieron antes: ¿tienen un **verificador externo confiable**?
2. Si no lo tienen, ¿pueden construir uno barato? (tests, reglas, una base de datos, un cálculo)
3. ¿Quién decidiría **cuándo parar** de iterar sin hacer trampa con la respuesta correcta?

Provocación

Si su respuesta al punto 2 es “no se puede”, quizá el problema no es el agente: es que la tarea aún no está bien definida.

Dónde estamos

El problema

¿De dónde viene la señal?

Los métodos, uno por uno

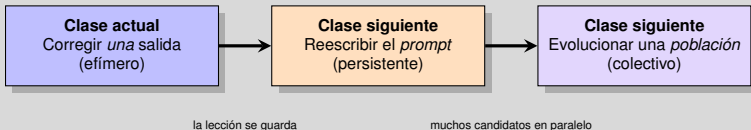
¿Funciona de verdad?

De corregir a mejorarse

Práctica

Cierre

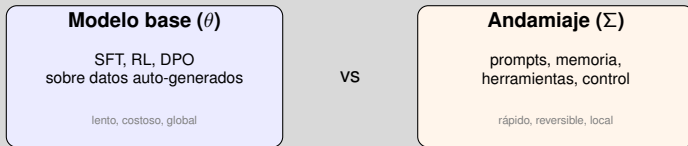
La corrección que se queda



Es el mismo mecanismo en tres escalas. Self-Refine y Reflexion son solo el primer escalón.

¿Y qué se puede modificar, exactamente, cuando un agente “se mejora”?

¿Qué puede cambiar un agente de sí mismo?



SFT (*Supervised Fine-Tuning*): reentrenar el modelo con ejemplos de entrada y salida correcta. **RL** (*Reinforcement Learning*): ajustar los pesos maximizando una recompensa numérica. **DPO** (*Direct Preference Optimization*): ajustar con pares “esta respuesta es mejor que aquella”, sin entrenar un modelo de recompensa aparte.

Casi todo lo accesible hoy está en Σ : **no hace falta reentrenar** para que el agente mejore.

Ren et al., “Self-Improvements in Modern Agentic Systems”, arXiv:2607.13104 (2026), §3.2.

Las clases 11 y 12 viven enteras dentro de esa caja naranja.

Discusión en grupo · 3 min

¿ θ o Σ ?

1. Para su tarea, ¿cambiarían el **modelo** (θ) o el **andamiaje** (Σ)? ¿Qué los limita: datos, dinero o permisos?
2. SCoRe necesitó reentrenar para que el Δ fuera positivo. ¿Podrían permitirselo?
3. Si solo pueden tocar Σ , ¿qué parte cambiarían primero: prompt, memoria o herramientas?

Regla del pulgar

Empieza siempre por Σ . Pasa a θ solo cuando el andamiaje ya no dé más de sí.

Dónde estamos

El problema

¿De dónde viene la señal?

Los métodos, uno por uno

¿Funciona de verdad?

De corregir a mejorarse

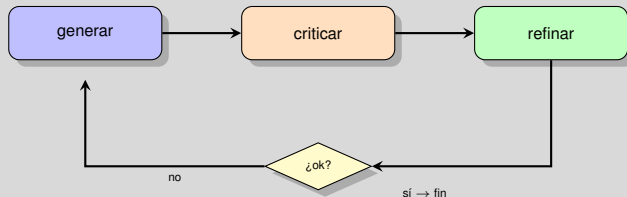
Práctica

Cierre

Práctica: construyamos el loop

Dos agentes: uno se juzga solo, el otro deja que lo juzgue el intérprete

Agente 1: Self-Refine (feedback interno)

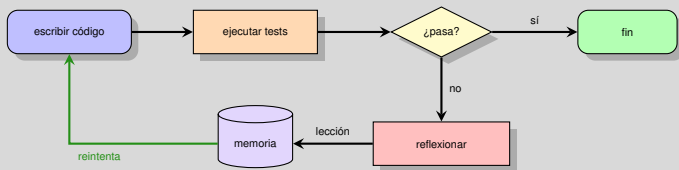


Es la **fila roja** del recap: barato y poco confiable. El propio LLM decide cuándo parar.

Ahora reemplacemos ese juez por uno que no puede alucinar.

`notebooks/10_feedback.ipynb` → Parte 1

Agente 2: Reflexion + intérprete (feedback externo)



Dos cambios respecto al agente 1: el juez es el **intérprete**, y la lección **persiste** entre problemas.

Con los dos montados, podemos probar experimentalmente lo que dijo Kamoi.

`notebooks/10_feedback.ipynb` → Parte 2

Experimento guiado

Mide, no supongas

1. Agente 1 con y sin la etapa de crítica → ¿mejora real o solo más tokens?
2. Agente 2 con memoria vaciada entre problemas → ¿sirve la memoria inter-tarea?
3. Quita el intérprete y deja que el LLM juzgue → ¿cuántas veces se equivoca?

Lo que deberías ver

El paso 3 reproduce el hallazgo de Kamoi et al.: cuando el juez falla, el loop optimiza hacia el lugar equivocado.

Cierra el círculo: volvemos al checklist con datos propios en la mano.

Dónde estamos

El problema

¿De dónde viene la señal?

Los métodos, uno por uno

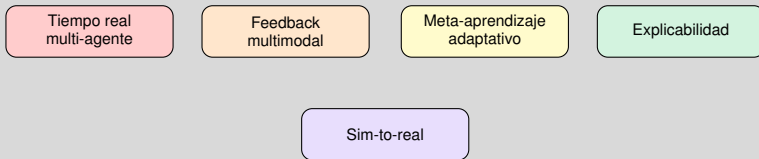
¿Funciona de verdad?

De corregir a mejorarse

Práctica

Cierre

Lo que queda abierto



Estandarización

Protocolos emergentes para agentes heterogéneos: **MCP**, **A2A**, ANP, Agora.

Liu et al., IJCAI-25, §6 (Challenges and Future Directions).

El primero de la lista (tiempo real multi-agente) es literalmente la clase 12.

Para llevarse

El hilo de hoy en cuatro frases

- El feedback convierte agentes **reactivos** en **adaptativos**
- Hay cuatro fuentes de señal: lo barato no es confiable, lo confiable no es barato
- Sin verificador externo, desconfía de la auto-corrección
- Corregir una salida es el primer escalón: sigue **reescribir el prompt**

Siguiente clase

Tenemos miles de trazas de agentes fallando. ¿Cómo las convertimos en un prompt mejor?

Referencias I



Liu, Z., Bai, X., Chen, K., et al. (2025). "A Survey on the Feedback Mechanism of LLM-based AI Agents." *IJCAI-25 Survey Track*, pp. 10582–10592.



Ren, Z., Chen, Y., Guo, D., et al. (2026). "Self-Improvements in Modern Agentic Systems: A Survey." arXiv:2607.13104.



Kamoi, R., Zhang, Y., Zhang, N., Han, J., Zhang, R. (2024). "When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs." *TACL*, 12:1417–1440.



Yao, S., Zhao, J., Yu, D., et al. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." *ICLR 2023*.



Shinn, N., Cassano, F., Berman, E., et al. (2023). "Reflexion: Language Agents with Verbal Reinforcement Learning." *NeurIPS 2023*.



Madaan, A., Tandon, N., Gupta, P., et al. (2023). "Self-Refine: Iterative Refinement with Self-Feedback." *NeurIPS 2023*.



Zhao, A., Huang, D., Xu, Q., et al. (2024). "ExpeL: LLM Agents are Experiential Learners." *AAAI 2024*.



Wei, J., Wang, X., Schuurmans, D., et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *NeurIPS 2022*.



Sun, H., Zhuang, Y., Kong, L., Dai, B., Zhang, C. (2023). "AdaPlanner: Adaptive Planning from Feedback with Language Models." *NeurIPS 2023*.



Fu, Y., Kim, D.-K., Kim, J., et al. (2024). "AutoGuide: Automated Generation and Selection of Context-Aware Guidelines for Large Language Model Agents." *NeurIPS 2024*.

Referencias II



Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023). "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena." *NeurIPS 2023*.



Zhuge, M., Zhao, C., Ashley, D., et al. (2024). "Agent-as-a-Judge: Evaluate Agents with Agents (DevAI)." arXiv:2410.10934.



Kumar, A., Zhuang, V., Agarwal, R., et al. (2025). "Training Language Models to Self-Correct via Reinforcement Learning (SCoRe)." *ICLR 2025 (Oral)*, Google DeepMind. arXiv:2409.12917.



Allard, M.-A., Teinturier, A., Xing, V., Viaud, G. (2026). "Experiential Reflective Learning for Self-Improving LLM Agents." arXiv:2603.24639.



Shi, Y., Li, S., Wu, C., Liu, Z., Fang, J., Cai, H., Zhang, A., Wang, X. (2025). "Search and Refine During Think: Facilitating Knowledge Refinement for Improved Retrieval-Augmented Reasoning (AutoRefine)." *NeurIPS 2025*. arXiv:2505.11277.



"Honest Lying: Understanding Memory Confabulation in Reflexive Agents" (2026). arXiv:2605.29463.



Wang, G., Xie, Y., Jiang, Y., et al. (2024). "Voyager: An Open-Ended Embodied Agent with Large Language Models." *TMLR*.



Du, Y., Li, S., Torralba, A., Tenenbaum, J., Mordatch, I. (2024). "Improving Factuality and Reasoning in Language Models through Multiagent Debate." *ICML 2024*.



Ouyang, L., Wu, J., Jiang, X., et al. (2022). "Training Language Models to Follow Instructions with Human Feedback." *NeurIPS 2022*.



Repositorio del survey: <https://github.com/kevinson7515/Agents-Feedback-Mechanisms>

Preguntas

Preguntas?

Dr. Vicente Machaca Arceda
vmachaca@utec.edu.pe